Solvark Database Design Document (DDD)

Version: 1.0 | Database: PostgreSQL (Supabase) | Architecture: Relational Database with Row Level Security (RLS)

SUMMARY

This document defines the database architecture, entities, relationships, constraints, indexing strategy, security model, and naming conventions for the Solvark platform.

PURPOSE

The schema is designed to support: Marketing website; Admin CMS; Portfolio management; Blog; Testimonials; Team management; Contact leads; Future CRM; Future Client Portal; Future Employee Portal

DATABASE OVERVIEW

Supabase PostgreSQL modules: Authentication, CMS, Portfolio, Blog, CRM, SEO, Settings, Analytics (future), Client Portal (future)

ENTITY RELATIONSHIP OVERVIEW

Users -> Projects -> Project Media; Users -> Blog Posts -> Categories, Tags; Users -> Testimonials; Users -> Services; Users -> Team Members; Users -> Homepage Sections; Clients -> Projects; Clients -> Contact Leads

CORE TABLES

users (id, email, full_name, avatar_url, role_id, created_at, updated_at); roles (id, name, description); permissions (examples: projects.create, projects.update, projects.delete, blogs.publish, seo.manage); role_permissions (bridge table, many-to-many Role ↔ Permission)

PORTFOLIO MODULE

clients (id, company_name, contact_name, email, phone, industry, status, notes, created_at); projects (id, client_id, title, slug, short_description, description, category, featured, live_url, github_url, completion_date, created_by, created_at); project_media (id, project_id, type IMAGE/VIDEO, url, alt_text, sort_order; one project to many media files); project_technologies stores React, Next.js, Node.js, Tailwind, Supabase (many-to-many relationship)

SERVICES MODULE

services (id, title, slug, icon, banner, description, featured, seo_title, meta_description); service_faqs (one service to many FAQs)

TESTIMONIALS

testimonials (id, client_name, company, designation, photo, rating, review, featured, published)

TEAM

team_members (id, name, role, photo, bio, linkedin, github, display_order)

BLOG

blog_posts (id, title, slug, content, excerpt, featured_image, category_id, author_id, published, published_at); blog_categories stores blog categories; blog_tags stores tags; blog_post_tags many-to-many bridge

CONTACT MODULE

contact_leads (id, name, company, email, phone, message, status, assigned_to, notes, created_at). Status: New, Contacted, Qualified, Closed

HOMEPAGE CMS

homepage_sections stores dynamic homepage content. Fields: section_key, title, subtitle, content, button_text, button_link, image_url, display_order, published. Examples: Hero, Services, Portfolio, CTA, FAQ

SEO

seo_metadata stores Page, Title, Description, Canonical, OG Image, Robots, JSON-LD

MEDIA LIBRARY

media_library stores File URL, Storage Path, MIME Type, Size, Uploaded By, Created At

WEBSITE SETTINGS

site_settings stores Logo, Favicon, Contact Info, Social Links, Footer, Theme, Analytics IDs

AUDIT LOGS

audit_logs tracks User, Action, Table, Record ID, Timestamp, IP Address (optional), User Agent (optional)

RELATIONSHIPS

roles -> users -> projects -> project_media -> clients; services -> service_faqs; blog_posts -> blog_categories; blog_posts -> blog_post_tags -> blog_tags; homepage_sections; seo_metadata; contact_leads; team_members; site_settings; media_library; audit_logs

INDEX STRATEGY

Indexes: email, slug, created_at, category_id, project_id, featured, published, status. Use composite indexes where filtering commonly combines fields (for example, published + created_at).

ROW LEVEL SECURITY (RLS)

Public: Read published website content only. Editor: Manage blogs, testimonials, projects. Admin: Full CMS access. Owner: Full system access.

SOFT DELETE STRATEGY

Every important table contains deleted_at and deleted_by. Records remain recoverable.

FUTURE TABLES

invoices, quotations, tasks, employee_profiles, client_portal_users, project_updates, project_files, notifications, activity_feed, crm_deals, crm_companies, crm_contacts, support_tickets, payments, subscriptions

NAMING STANDARDS

Tables: snake_case. Columns: snake_case. Primary Keys: id. Foreign Keys: *_id. Timestamps: created_at, updated_at, deleted_at.

DATA INTEGRITY

UUID primary keys; foreign key constraints; unique slugs; NOT NULL where appropriate; CHECK constraints for enums and ratings; ON DELETE rules chosen per relationship (CASCADE only where safe).

BACKUP & RECOVERY

Daily automated backups; point-in-time recovery (where available); monthly restore testing; versioned object storage for uploaded media.

DEFINITION OF DONE

The database is complete when: all core entities are modeled; relationships and constraints are enforced; indexes exist for common queries; RLS policies are defined; soft deletes are supported; schema is ready for migration tooling. Next Document: API Specification (OpenAPI) followed by the UI/UX Design Specification.